Distributed Training

CS6501: AI Systems

Fall 2026

Lecture 8

Yue Cheng



Material taken/derived from:

- Harvard ML Systems by Vijay Janapa Reddi and UCSD CSE 291P by Yufei Ding

@ 2026 released for use under a CC BY-SA license.

A training deadline

- 405B parameters
- 15T training tokens
- Deadline: about four months

Question: How many GPUs would we need?

Training work: C = 6ND

N = parameters D = training tokens

Forward ≈ 2ND	+	Backward ≈ 4ND

$6 \times (405 \times 10^9) \times (15 \times 10^{12})$

$\approx 3.645 \times 10^{25}$ FLOPs

Dense-model approximation. Attention and recomputation can add work.

Work divided by throughput

Assume 400 TFLOP/s of useful work per GPU

T ≈ total FLOPs / (GPUs × useful FLOP/s)

GPUs	Estimated time
1	≈ 2,890 years
16,000	≈ 66 days

The challenge is sustaining useful throughput.

Illustrative rate, not a measured LLM training run.

The memory constraint

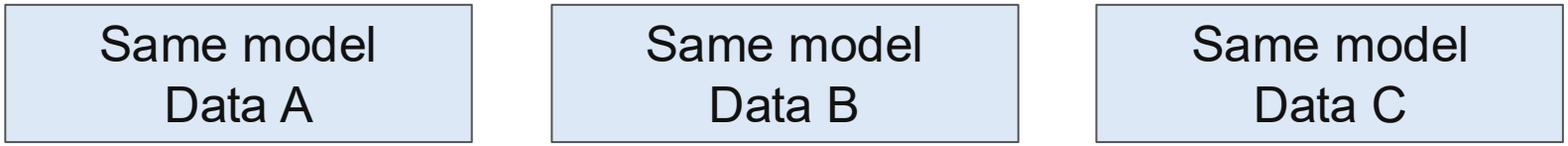
Training state	Bytes / parameter
Working weights + gradients	2 + 2
FP32 master weights + Adam moments	4 + 4 + 4
Total persistent state	16

405B × 16 bytes = 6.48 TB

Plus activations and temporary buffers

Three ways to split training

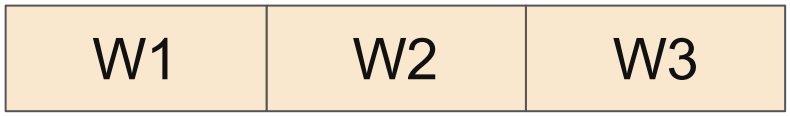
DP (data parallelism): split training samples



PP (pipeline parallelism): split model layers



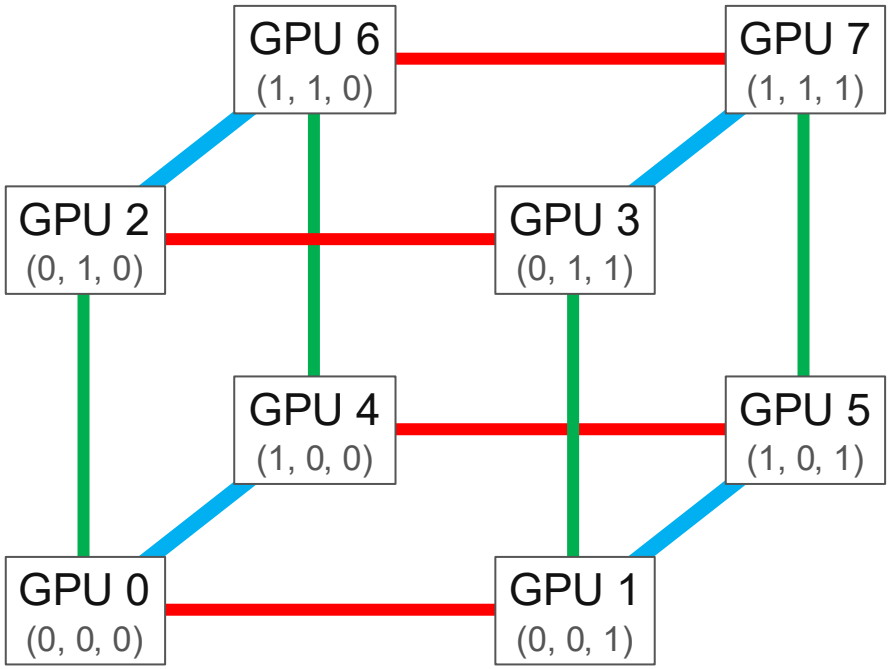
TP (tensor parallelism): split one layer's matrix



GPUs = d × p × t

The 3D parallelism cube

2 data replicas × 2 pipeline stages × 2 tensor shards = 8 GPUs



DP: split examples

Same model shard, different data

PP: split layers

Different stages of the same model

TP: split a layer

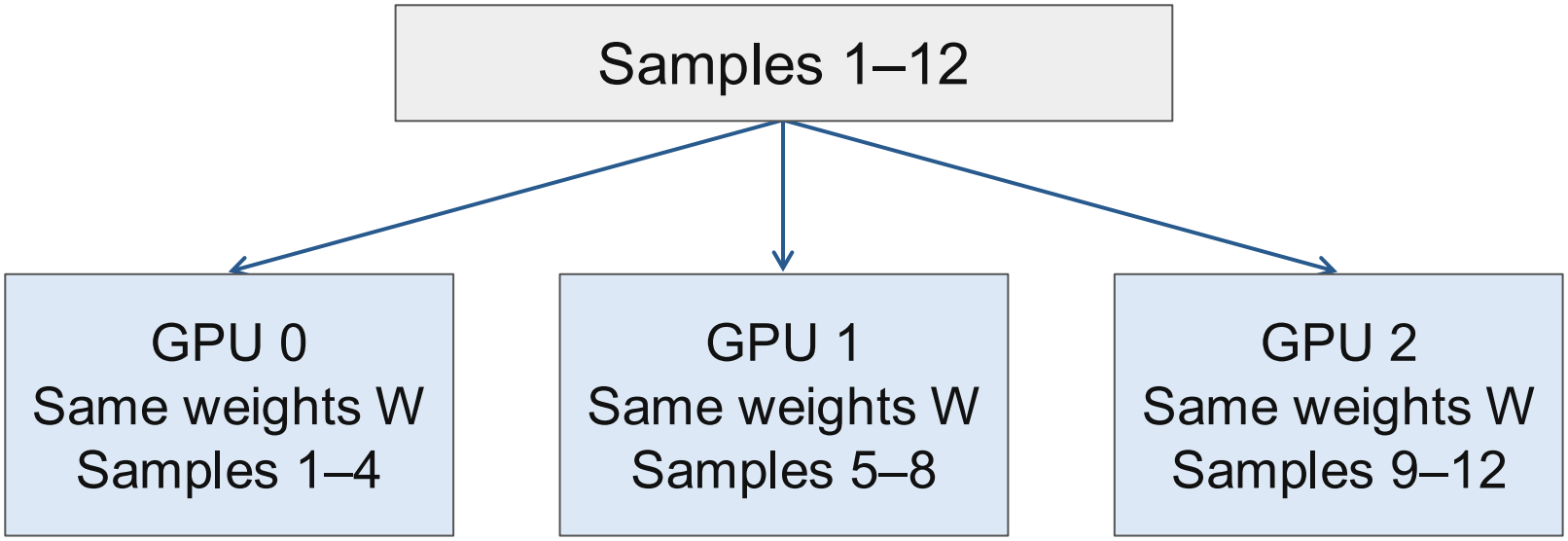
Different matrix shards, same input

Front and back squares: two model replicas

Coordinates: (data replica, pipeline stage, tensor shard). Edges show logical groups.

Data parallelism

One global batch of 12 samples



Different data, identical starting parameters

Why average gradient?

Start: w = 10 Learning rate: η = 0.1

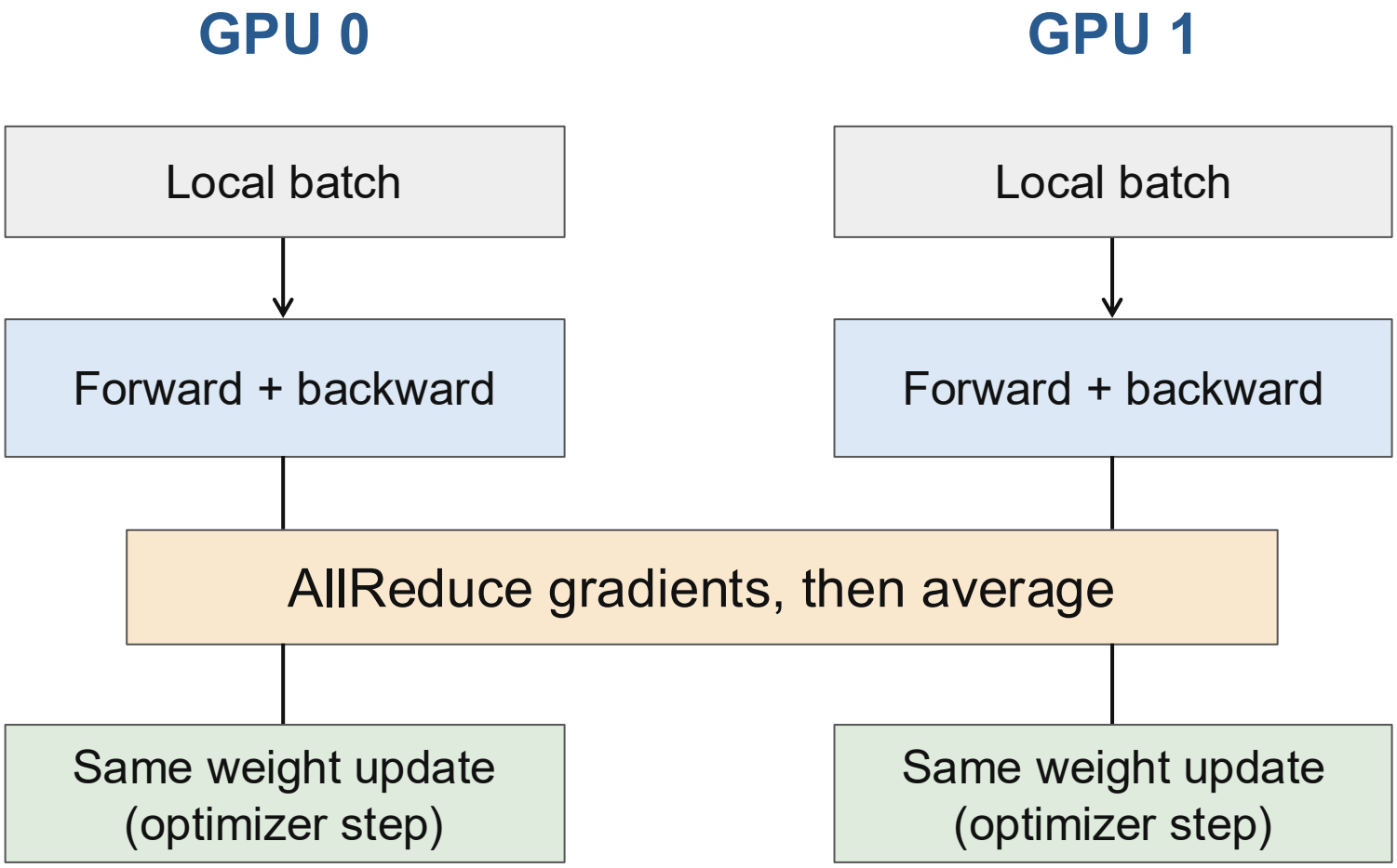
GPU 0	GPU 1
$g_0 = 2$	$g_1 = 6$

Mean gradient: (2 + 6) / 2 = 4

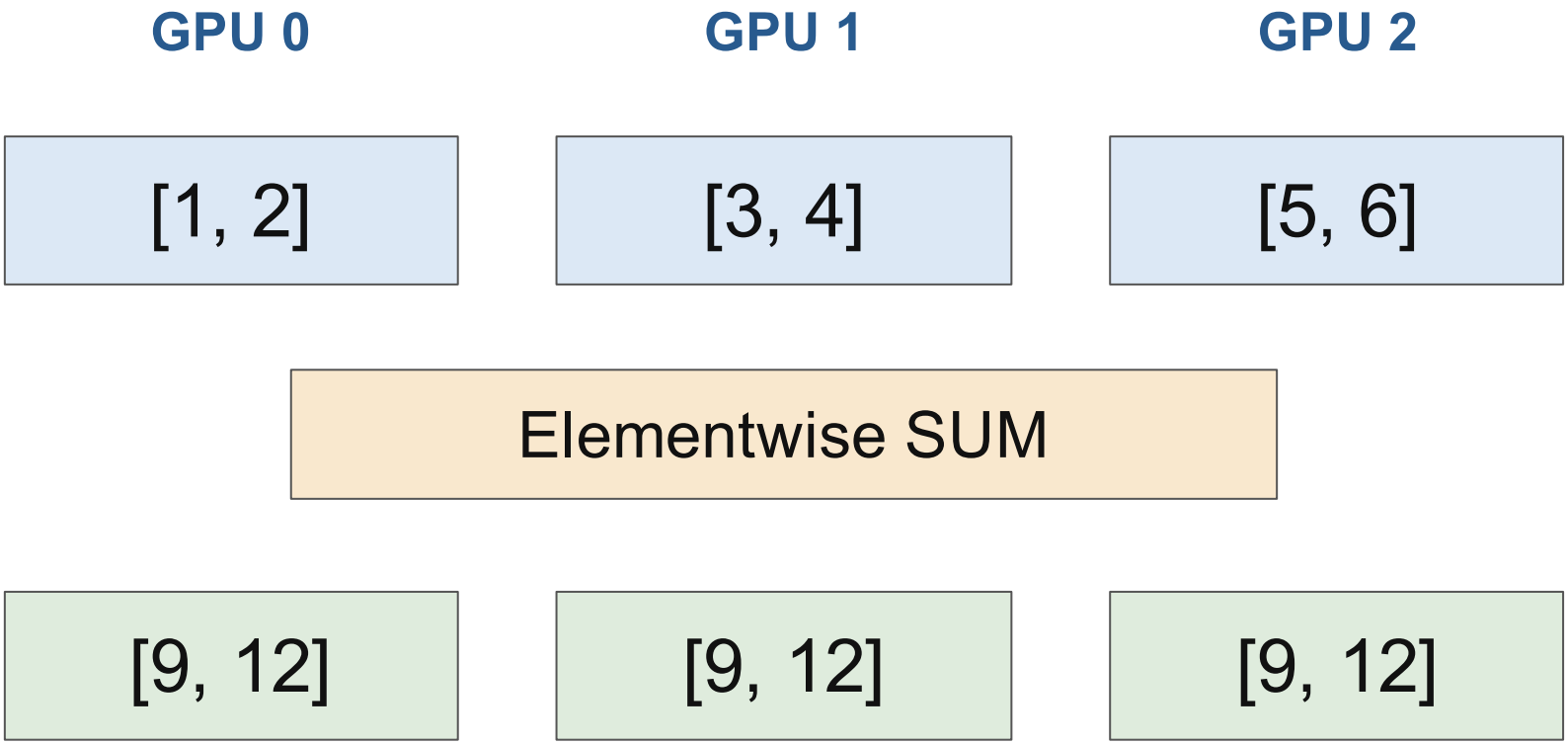
SGD optimizer step (both GPU):
$w = 10 - 0.1 \times 4 = 9.6$

Equal local batch sizes and mean losses.

One synchronized DP step



AllReduce



Every GPU receives the same result

For mean gradients: [9, 12] / 3 = [3, 4].

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

After round	GPU 0	GPU 1	GPU 2
Start	A_0, B_0, C_0	A_1, B_1, C_1	A_2, B_2, C_2
1: receive + add	$C_2 + C_0$	$A_0 + A_1$	$B_1 + B_2$
2: receive + add	$B_0+B_1+B_2$	$C_0+C_1+C_2$	$A_0+A_1+A_2$

Each GPU owns one fully summed chunk

Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

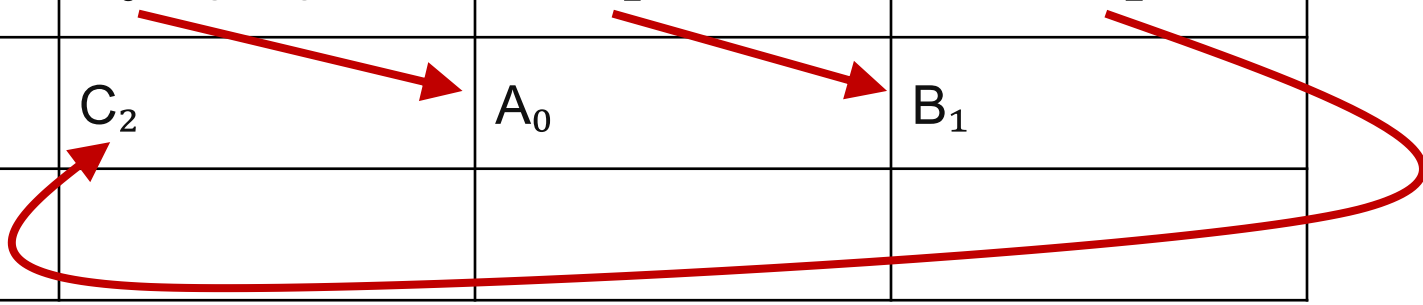
After round	GPU 0	GPU 1	GPU 2
Start	**A_0**, B_0, C_0	A_1, **B_1**, C_1	A_2, B_2, **C_2**

Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

After round	GPU 0	GPU 1	GPU 2
Start	**A_0**, B_0, C_0	A_1, **B_1**, C_1	A_2, B_2, **C_2**
1: receive	C_2	A_0	B_1



Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

After round	GPU 0	GPU 1	GPU 2
Start	A_0, B_0, C_0	A_1, B_1, C_1	A_2, B_2, C_2
1: receive + add	$C_2 + C_0$	$A_0 + A_1$	$B_1 + B_2$

Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

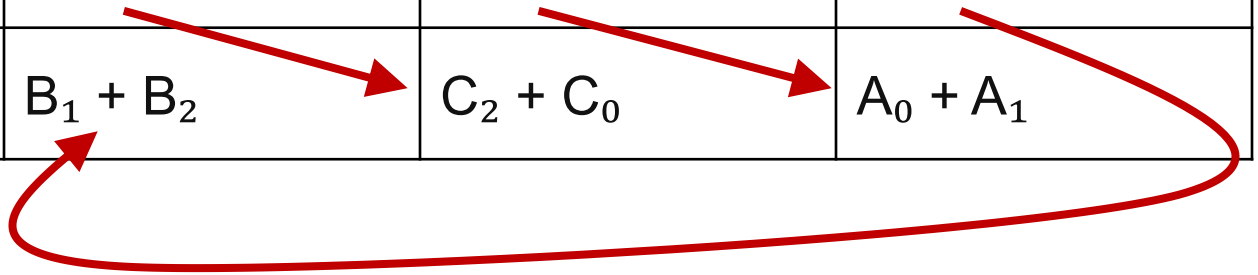
After round	GPU 0	GPU 1	GPU 2
Start	A_0, B_0, C_0	A_1, B_1, C_1	A_2, B_2, C_2
1: receive + add	**$C_2 + C_0$**	**$A_0 + A_1$**	**$B_1 + B_2$**

Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

After round	GPU 0	GPU 1	GPU 2
Start	A_0, B_0, C_0	A_1, B_1, C_1	A_2, B_2, C_2
1: receive + add	**$C_2 + C_0$**	**$A_0 + A_1$**	**$B_1 + B_2$**
2: receive	$B_1 + B_2$	$C_2 + C_0$	$A_0 + A_1$



Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

After round	GPU 0	GPU 1	GPU 2
Start	A_0	B_1	C_2
1: receive + add	$C_2 + C_0$	$A_0 + A_1$	$B_1 + B_2$
2: receive + add	**$B_0+B_1+B_2$**	**$C_0+C_1+C_2$**	**$A_0+A_1+A_2$**

Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: reduce-scatter

Ring direction: GPU 0 → GPU 1 → GPU 2 → GPU 0

After round	GPU 0	GPU 1	GPU 2
Start	A_0	B_1	C_2
1: receive + add	$C_2 + C_0$	$A_0 + A_1$	$B_1 + B_2$
2: receive + add	$\mathbf{B_0+B_1+B_2}$	$\mathbf{C_0+C_1+C_2}$	$\mathbf{A_0+A_1+A_2}$

↓ ↓ ↓

State after reduce-scatter: **B** **C** **A**

Each GPU owns one fully summed chunk

Each round sends one chunk per GPU. Table tracks the arriving chunk only.

Ring: all-gather

$A = A_0 + A_1 + A_2$ B and C likewise

After round	GPU 0	GPU 1	GPU 2
Start	B	C	A
1: receive	B, A	C, B	A, C
2: receive	A, B, C	A, B, C	A, B, C

AllReduce = reduce-scatter + all-gather

No arithmetic in all-gather. Divide by the worker count for the mean.

Ring: all-gather

$A = A_0 + A_1 + A_2$ B and C likewise

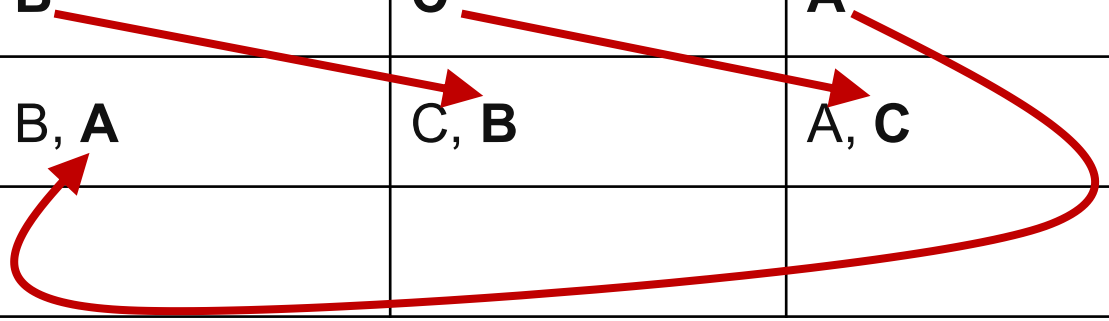
After round	GPU 0	GPU 1	GPU 2
Start	B	C	A

No arithmetic in all-gather. Divide by the worker count for the mean.

Ring: all-gather

$A = A_0 + A_1 + A_2$ B and C likewise

After round	GPU 0	GPU 1	GPU 2
Start	**B**	**C**	**A**
1: receive	B, **A**	C, **B**	A, **C**

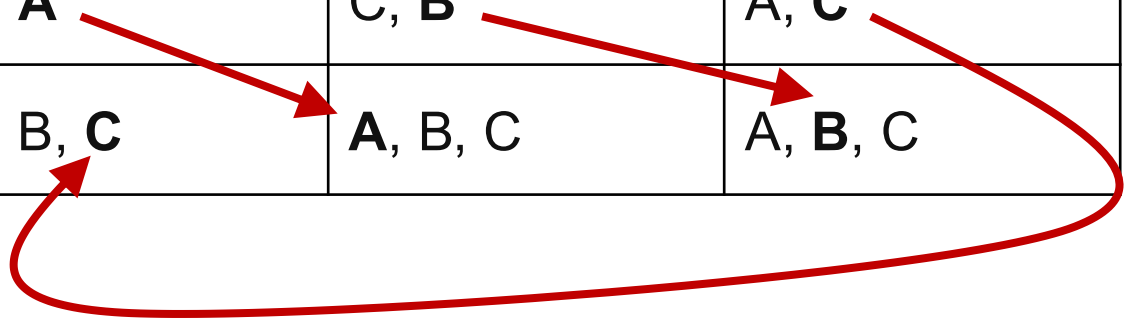


No arithmetic in all-gather. Divide by the worker count for the mean.

Ring: all-gather

$A = A_0 + A_1 + A_2$ B and C likewise

After round	GPU 0	GPU 1	GPU 2
Start	B	C	A
1: receive	B, **A**	C, **B**	A, **C**
2: receive	A, B, **C**	**A**, B, C	A, **B**, C



AllReduce = reduce-scatter + all-gather
Ring-AllReduce = Ring direction + AllReduce

No arithmetic (sum) in all-gather. Divide by the worker count for the mean.

Ring communication cost

n workers, G bytes of gradients, B bytes/s

2(n − 1) rounds × G/n bytes per round

$T \approx 2(n-1)\alpha + 2(n-1)G / (nB)$

n = 8, G = 1 GB, B = 25 GB/s

Payload time ≈ 1.75GB / 25 GB/s = 70 ms

Ideal ring model. α = startup time per round. Ignore contention here.

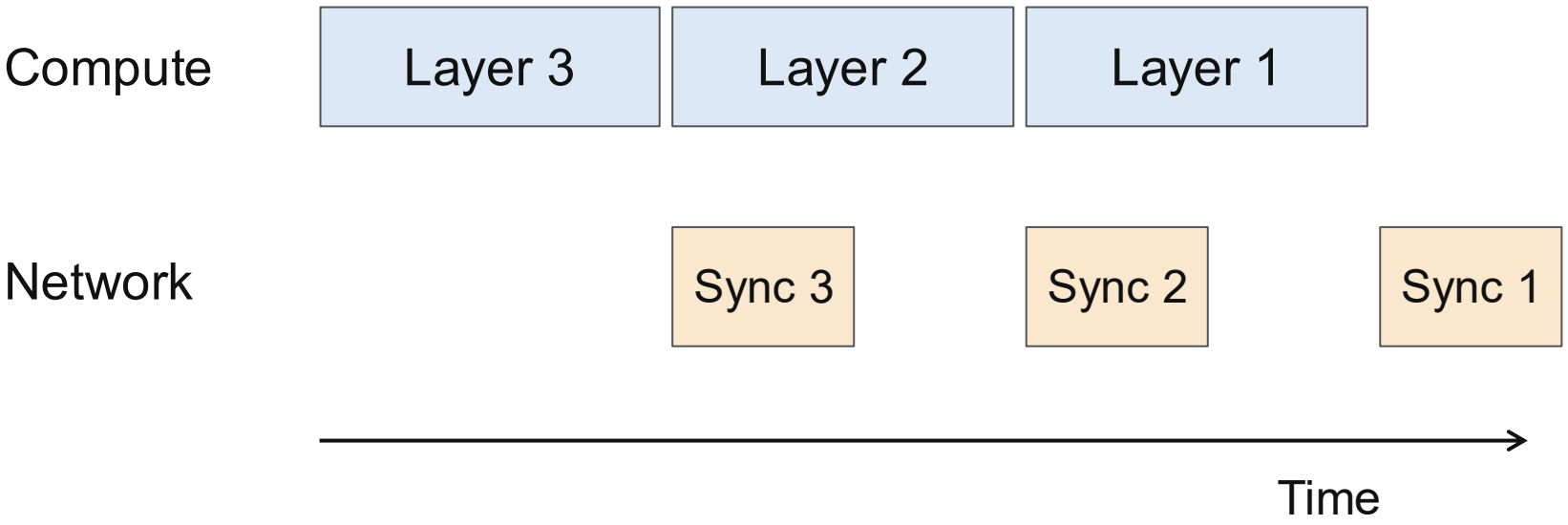
Question: Small msg-dominant workload

DP: Which term hurts for many small messages?

Specifically, for a fixed GPU count and total gradient size, why can many small, sequential AllReduce calls cost more than one large AllReduce?

Hiding gradient exchange

Backward produces gradients in reverse layer order

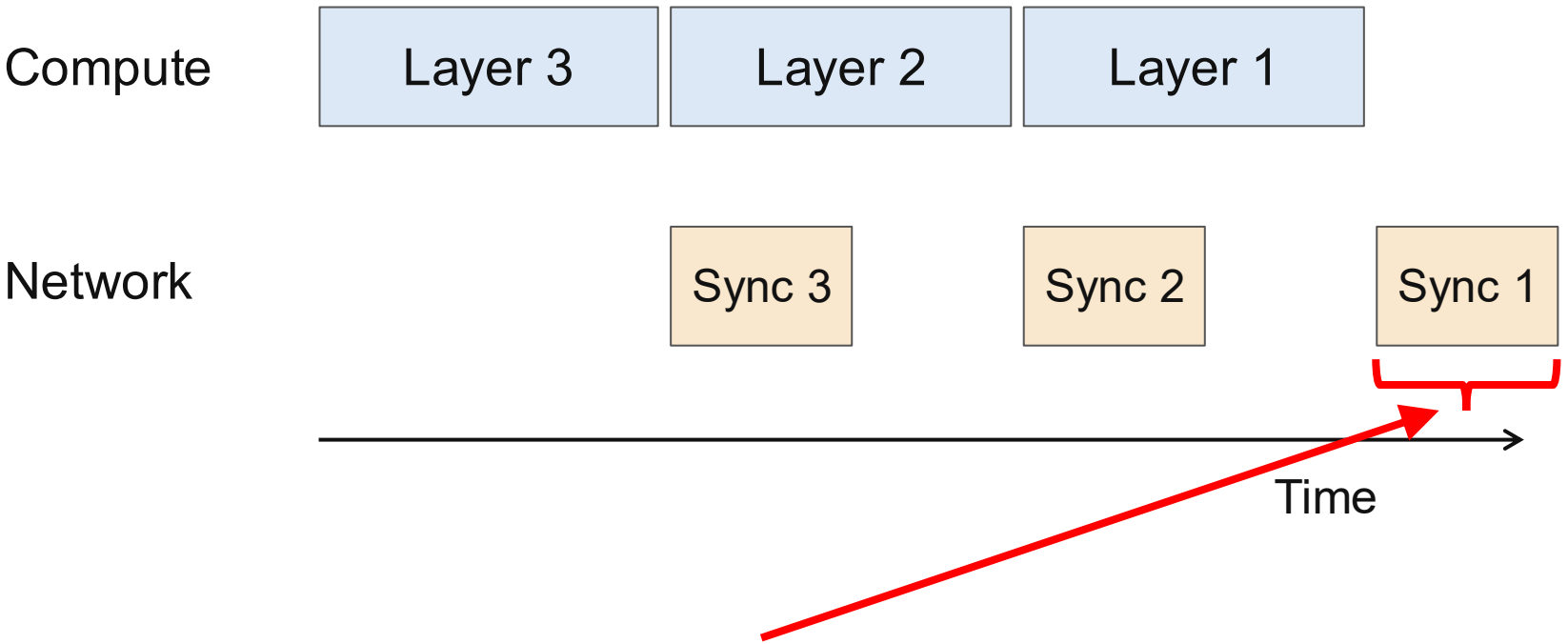


Only exposed communication extends the step

Each synchronization waits for its gradients on all workers.

Hiding gradient exchange

Backward produces gradients in reverse layer order



Only exposed communication extends the step

Each synchronization waits for its gradients on all workers.

Why DP stops scaling

Fixed global batch; illustrative timings (exposed sync at 60 ms is optimistic when scaling out)

GPUs	Compute	Exposed sync	Step time
4	200 ms	60 ms	260 ms
8	100 ms	60 ms	160 ms
16	50 ms	60 ms	110 ms

16 GPUs: 800 / 110 ≈ 7.3× speedup

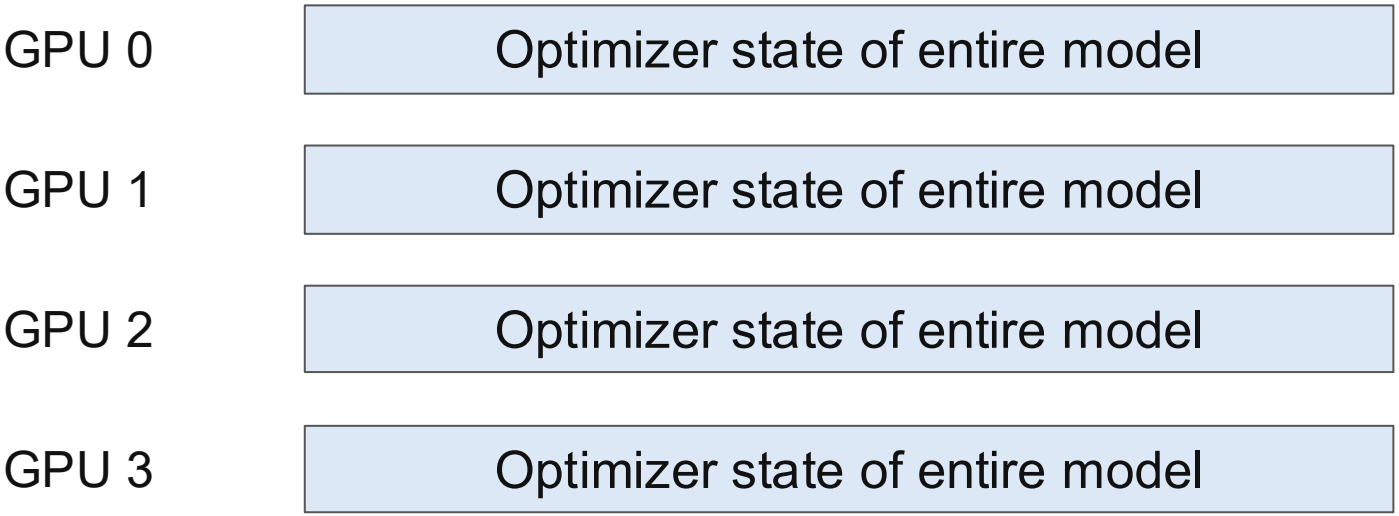
Assumed single-GPU compute: 800 ms. This is not a benchmark.

Question: What changes?

DP: If we double the local batch instead, what changes?

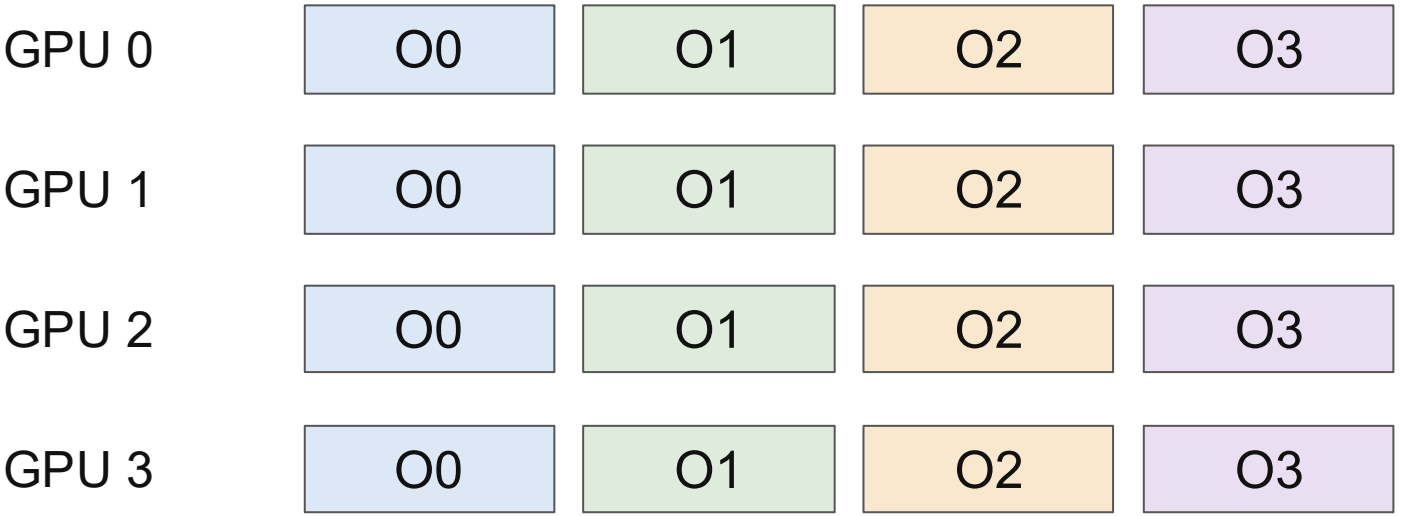
DP repeats training state

Replicated optimizer state



DP repeats training state

Replicated optimizer state

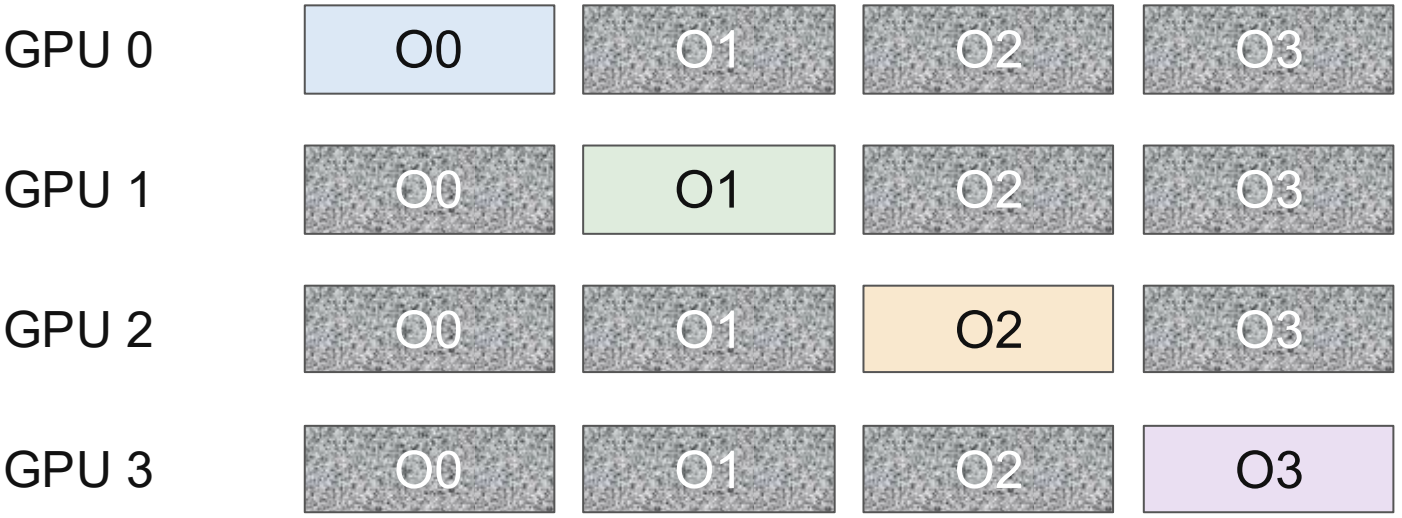


ZeRO: each GPU can own a different shard

Rajbhandari et al. ZeRO: Memory optimizations toward training trillion parameter models. https://arxiv.org/abs/1910.02054

DP repeats training state

Replicated optimizer state



ZeRO: each GPU can own a different shard

Rajbhandari et al. ZeRO: Memory optimizations toward training trillion parameter models. https://arxiv.org/abs/1910.02054

ZeRO stages

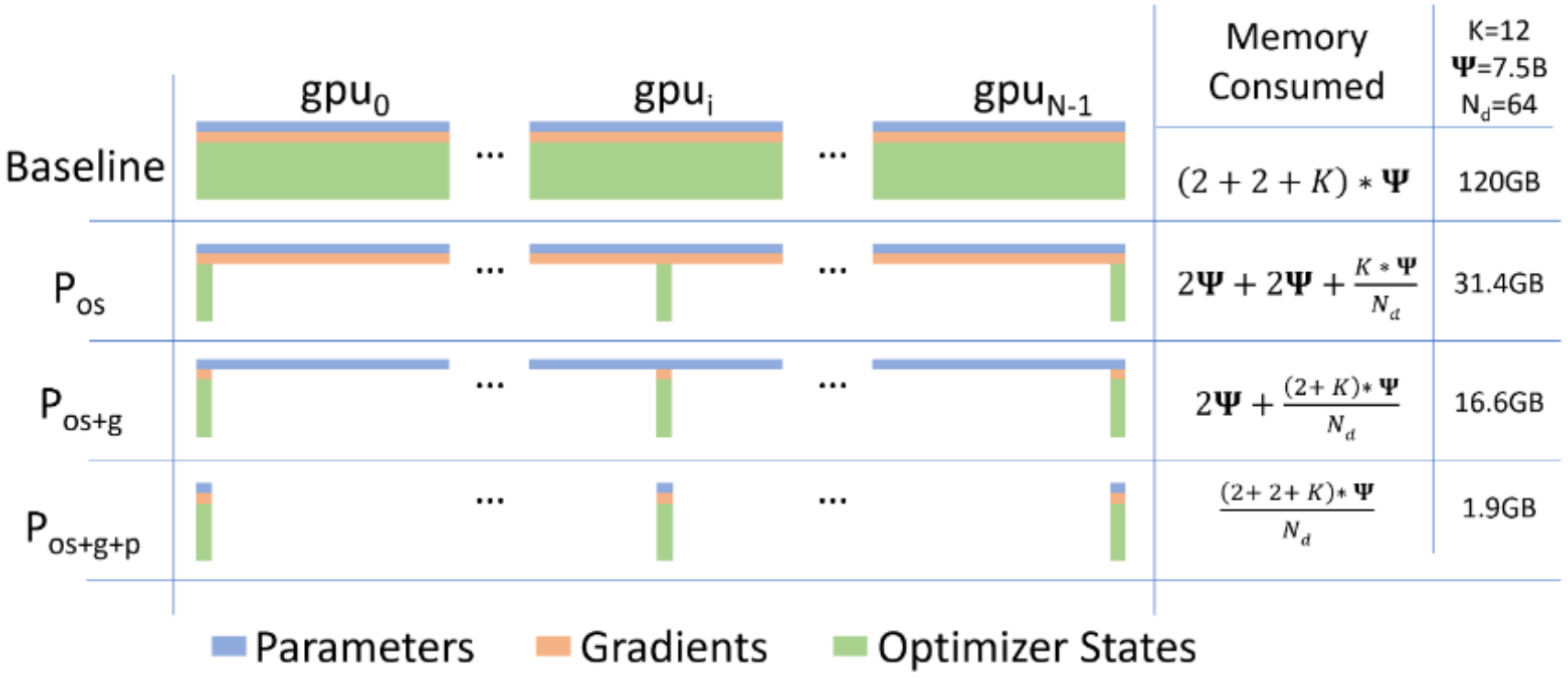
Persistent state	Basic DP	ZeRO-1	ZeRO-2	ZeRO-3
Optimizer	Full	Shard	Shard	Shard
Gradients	Full	Full	Shard	Shard
Parameters	Full	Full	Full	Shard

More sharding saves memory but changes communication

ZeRO-3 temporarily gathers parameters needed for computation.

Rajbhandari et al. ZeRO: Memory optimizations toward training trillion parameter models. https://arxiv.org/abs/1910.02054

ZeRO stages

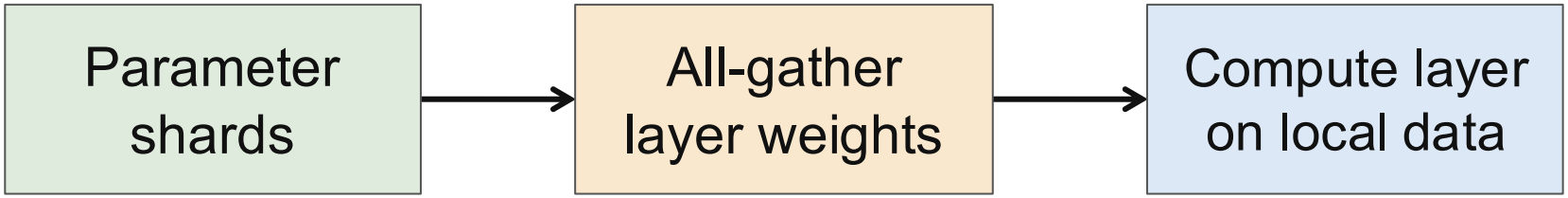


More sharding saves memory but changes communication

ZeRO-3 temporarily gathers parameters needed for computation.

Rajbhandari et al. ZeRO: Memory optimizations toward training trillion parameter models. https://arxiv.org/abs/1910.02054

ZeRO-3 during execution



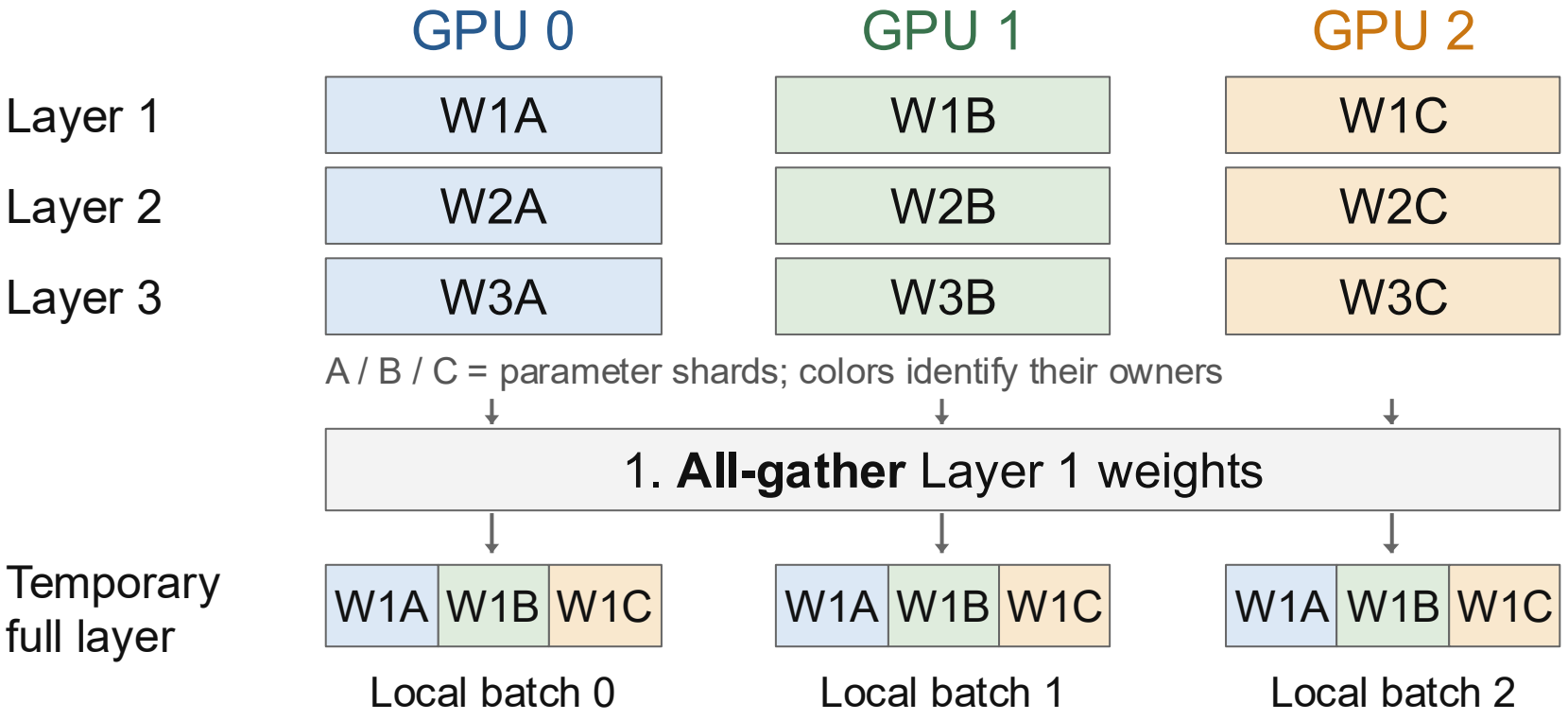
Release gathered weights after use

Backward: gather weights, reduce-scatter gradients

Shard storage, reconstruct weights when needed

ZeRO-3: Gather weights to compute

3 GPUs, 3 layers. Each GPU owns one third of every layer.



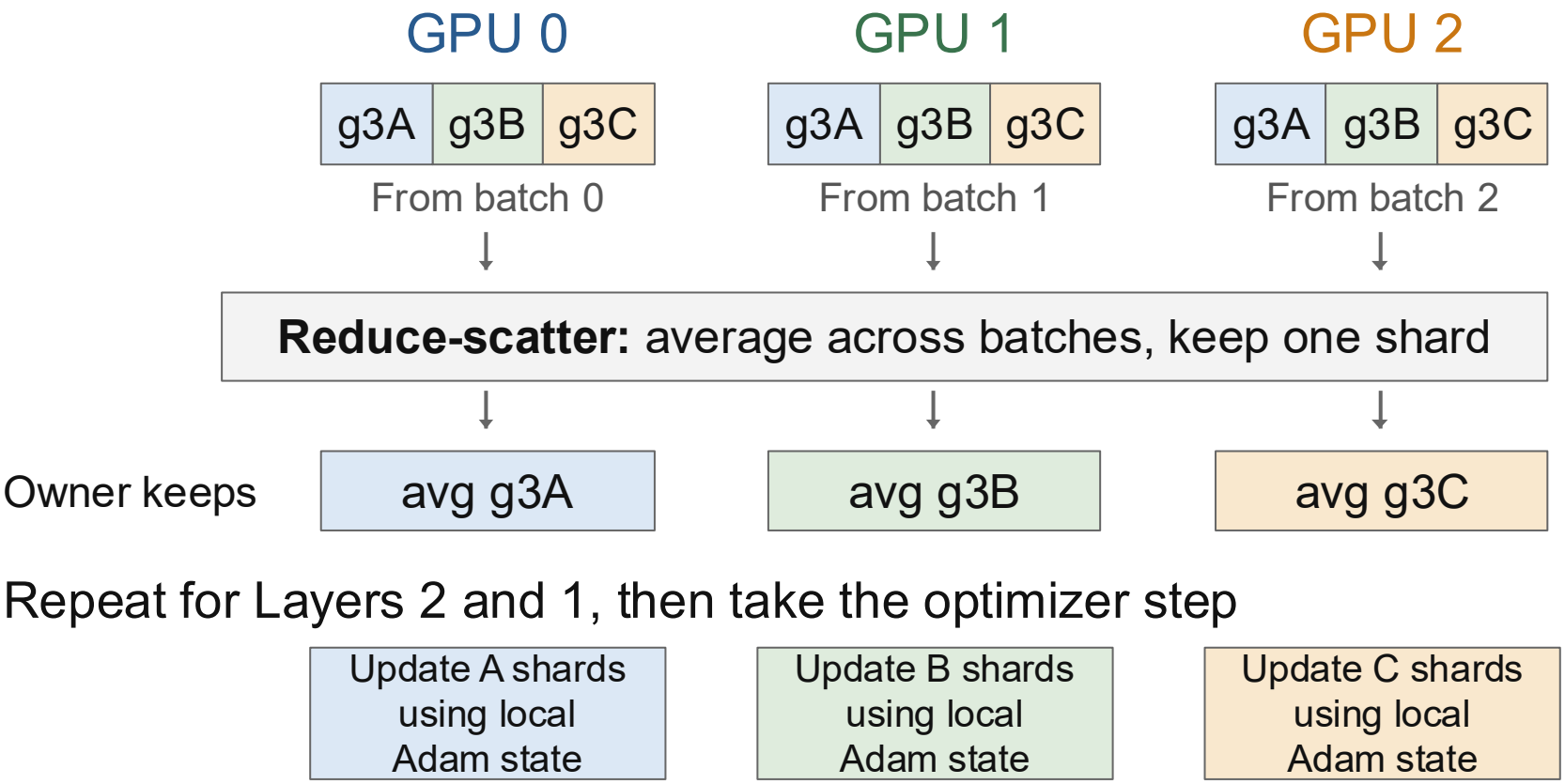
2. Keep owned shard; save computed activations; release temporary copies

Repeat for Layer 2, then Layer 3

ZeRO-3: Reduce gradients, update

Backward visits Layer 3, then Layer 2, then Layer 1

For Layer 3: all-gather weights again, compute local gradients using activations

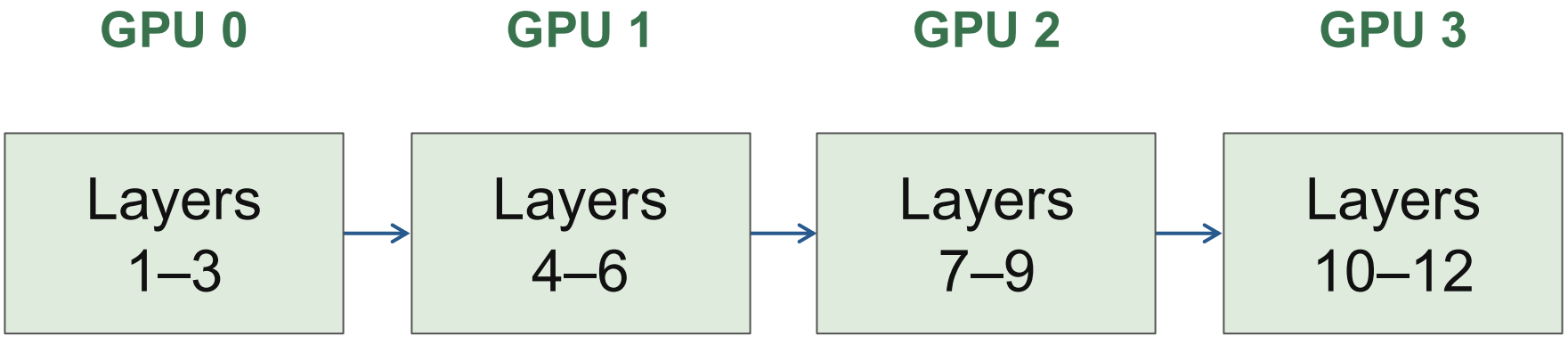


Repeat for Layers 2 and 1, then take the optimizer step

Update A shards using local Adam state	Update B shards using local Adam state	Update C shards using local Adam state

Next forward all-gathers the updated weights

Pipeline parallelism



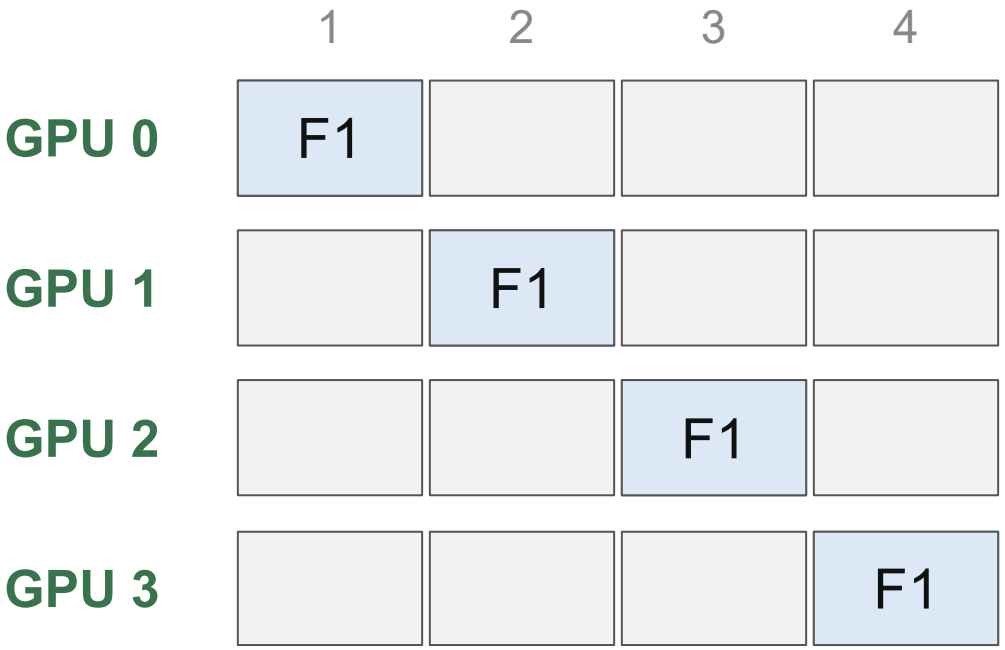
Forward: activations

Backward: activation gradients travel back

The model is partitioned by layers across GPUs
Each GPU owns a different part of the model

One batch leaves GPUs idle

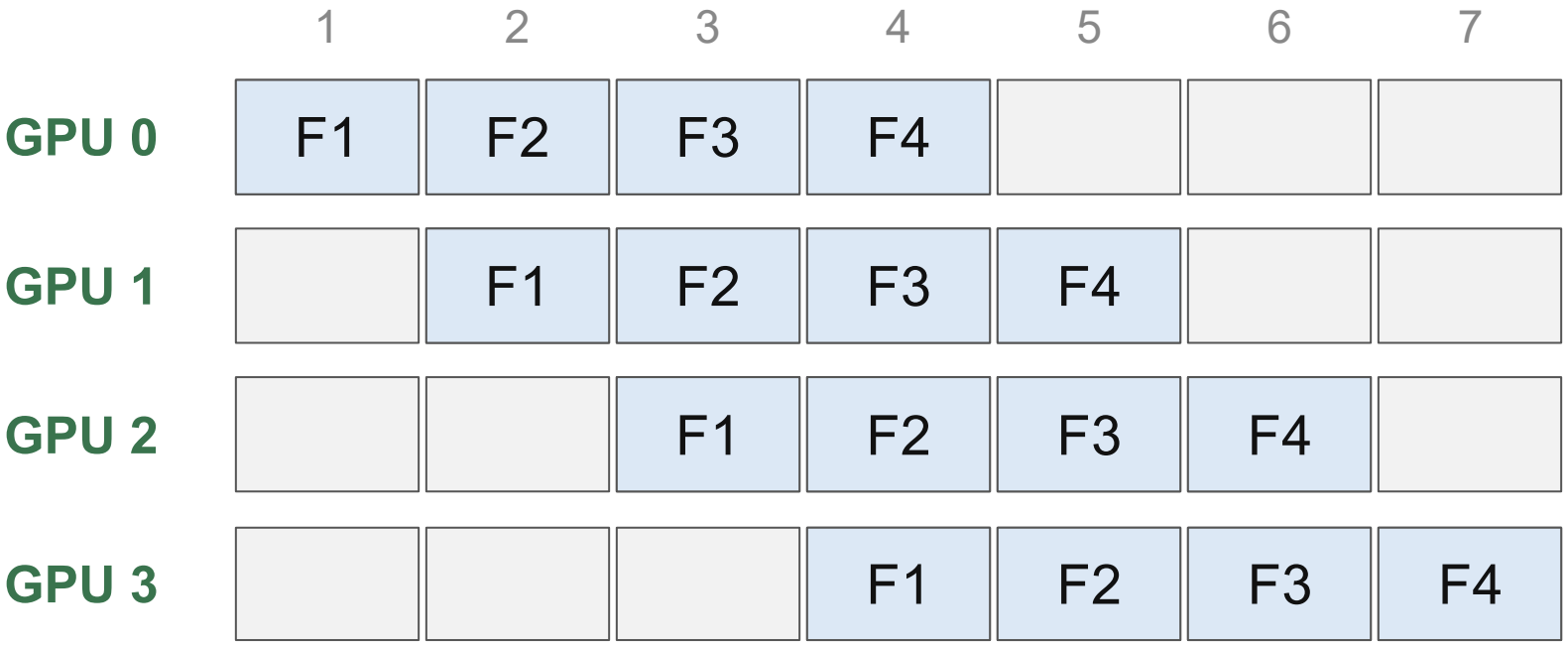
Forward only. One batch, four equal stages.



Only one stage works at a time

Microbatches fill the pipeline

F1 = forward pass of microbatch 1

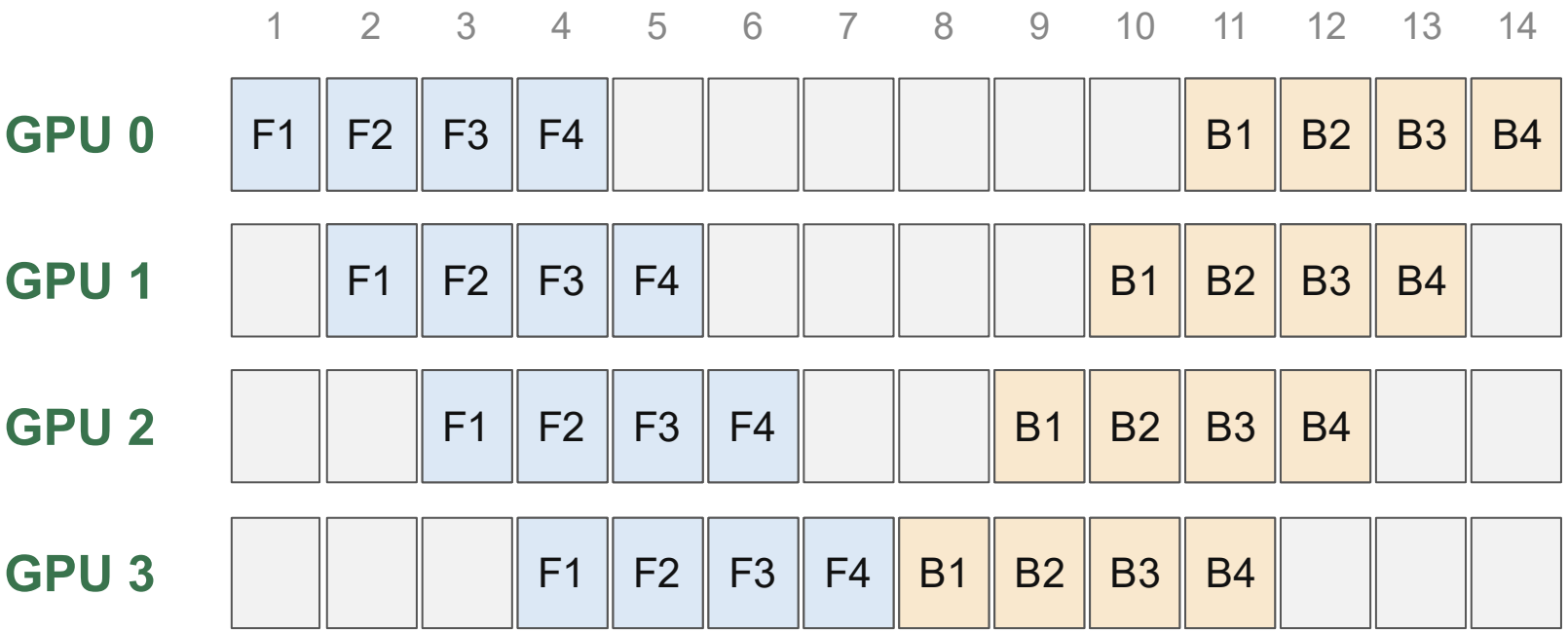


Different microbatches occupy different stages

Four microbatches belong to one local batch and one optimizer update.

Forward, backward, update

Simple GPipe-style schedule: all forward, all backward



Accumulate all microbatch gradients, then update

F = forward, B = backward. Equal slot lengths here are illustrative.

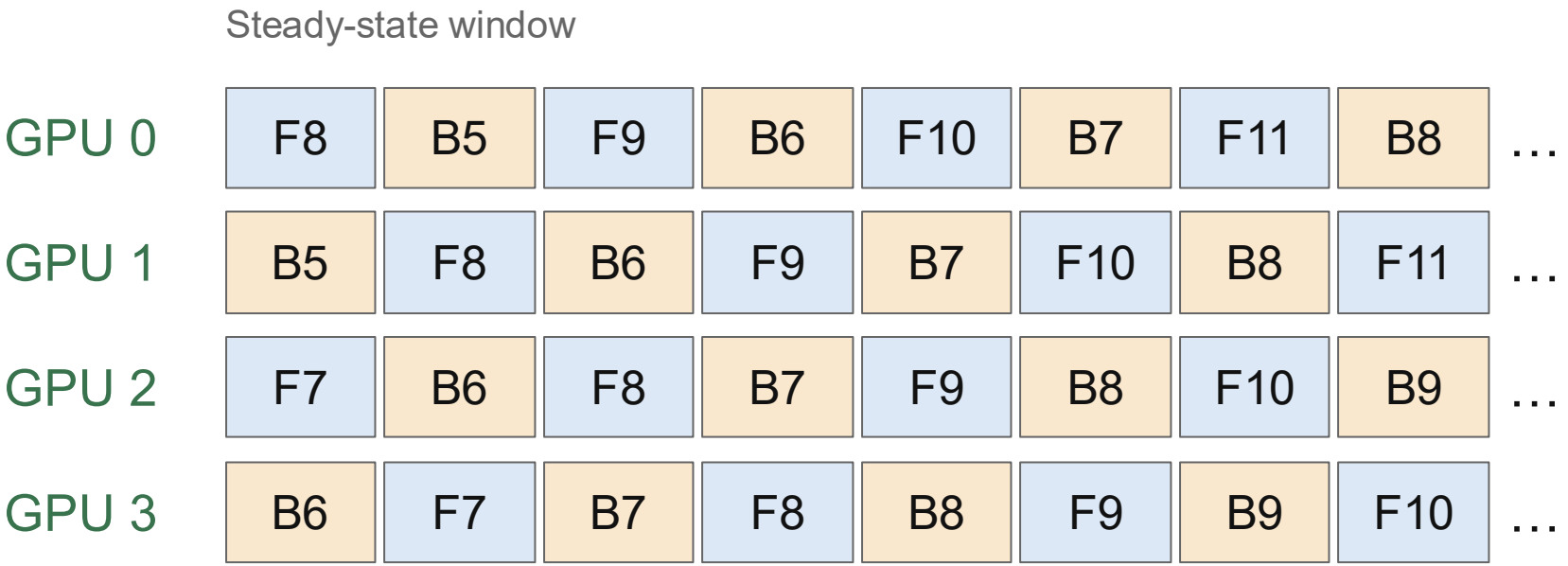
Huang et al, GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. https://arxiv.org/abs/1811.06965

Question: Why not update weights after B1 completes?

Synchronous PP: After B1 finishes, why accumulate its gradients instead of immediately updating weights?

PipeDream: A continuous pipeline

After warm-up: one forward, one backward (1F1B)



Each stage updates after backward. Other batches keep moving.

No repeated drain-and-refill at every update

F/B: forward/backward for batch i. Balanced stages, equal slots, negligible communication.

Narayanan et al., PipeDream: Generalized Pipeline Parallelism for DNN Training, SOSP 2019.

PipeDream: Weight stashing

A batch returns for backward after weights have changed

Zoom in on GPU 0 from the previous slide



Apply g8 to the latest weights: W7 → W8

Tradeoff: stale gradients + extra weight copies

Narayanan et al. PipeDream, SOSP 2019. Forward/backward versions match within each stage.

The pipeline bubble

p stages, m microbatches

Utilization ≈ m / (m + p − 1)

Idle fraction ≈ (p − 1) / (m + p − 1)

Stages	Microbatches	Idle fraction
4	4	3/7 ≈ 43%
4	16	3/19 ≈ 16%

Balanced stages, negligible communication. AFAB duration: (m+p−1)(f+b).

PP communication and memory

One boundary tensor: b × S × H elements

1 × 2,048 × 4,096 × 2 bytes

≈ 16.8 MB per microbatch, per direction

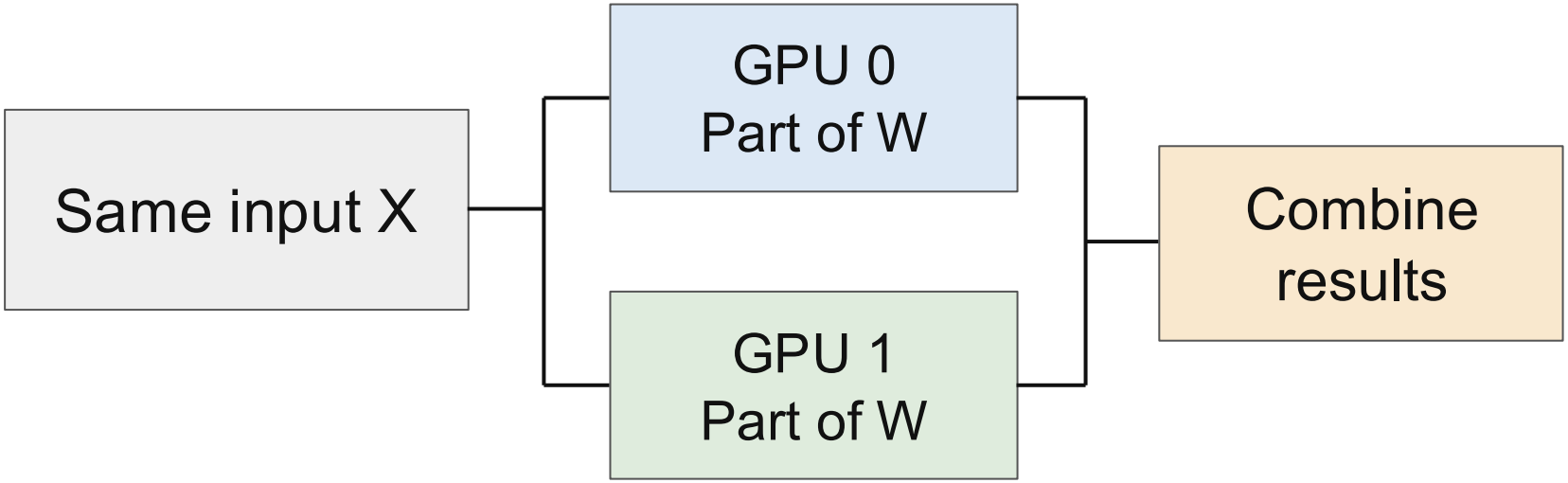
Saved activations consume memory until backward

1F1B frees activations sooner

1F1B interleaves forward/backward. It does not automatically remove bubbles.

Tensor parallelism

One layer: Y = XW



Same tokens, different pieces of one operation

Split output columns

$$W = \begin{bmatrix} 1 & 2 & 5 & 6 \\ 3 & 4 & 7 & 8 \end{bmatrix}$$

$X = [1 \;\; 2]$

GPU 0: W_0

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

GPU 1: W_1

$$\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$XW_0 = [7 \;\; 10]$

$XW_1 = [19 \;\; 22]$

Concatenate: Y = [7 10 19 22]

Each GPU computes complete values for a subset of output features.

Split the reduction dimension

Split input features to match the weight rows

$X = [X_0 \;\; X_1] = [1 \;\; 2]$

W

1	2	5	6
3	4	7	8

GPU 0: W_0

$1 \times [1 \;\; 2 \;\; 5 \;\; 6]$

GPU 1: W_1

$2 \times [3 \;\; 4 \;\; 7 \;\; 8]$

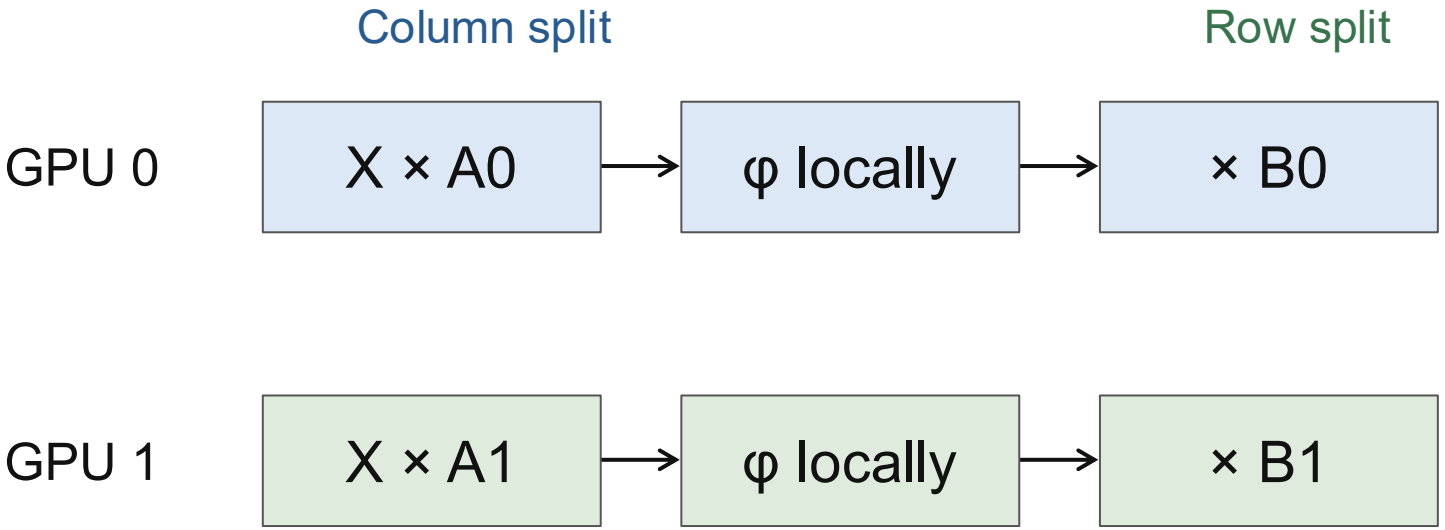
$[1 \;\; 2 \;\; 5 \;\; 6] \quad + \quad [6 \;\; 8 \;\; 14 \;\; 16]$

Sum partial results: Y = [7 10 19 22]

AllReduce combines partial sums when both GPUs need the full output.

A tensor-parallel MLP

$Y = \varphi(XA)B$



$$Y = \varphi(XA_0)B_0 + \varphi(XA_1)B_1$$

One output AllReduce, no intermediate gather

TP in a transformer

Same tokens on both GPUs

GPU 0
Attention heads 1–4

GPU 1
Attention heads 5–8

Output projection combines head contributions

Communication repeats inside every block

Basic Megatron-style TP has attention and MLP reductions in forward.

The “Jeff Dean test”

Thought experiment: one 100 MB payload

Effective bandwidth	Payload time
400 GB/s	0.25 ms
25 GB/s	4 ms

If layer compute takes 1 ms, the difference matters.

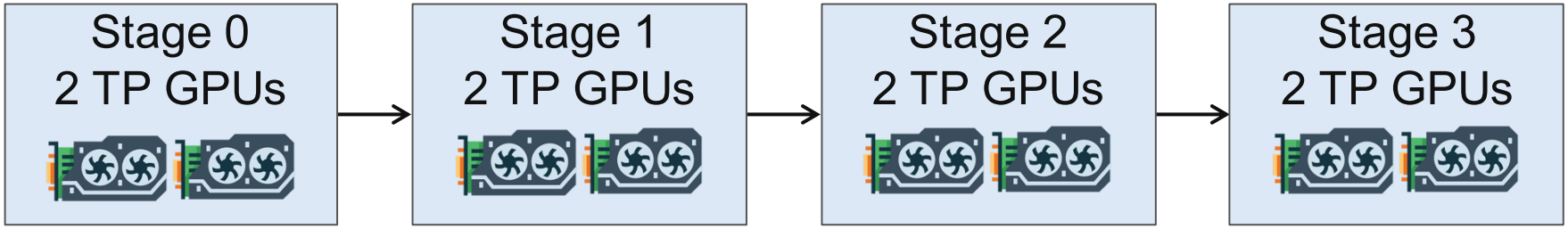
Frequent communication needs a fast interconnect

Payload-only estimates. Collectives also have startup and algorithm costs.

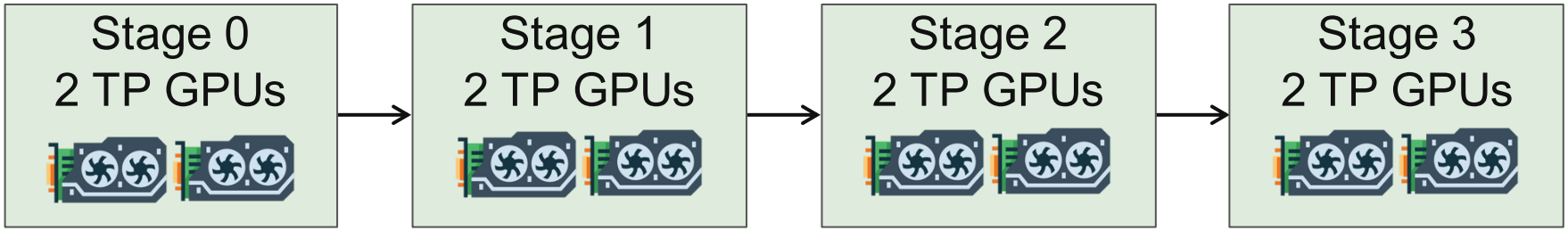
Combining all three

t = 2 p = 4 d = 2 GPUs = 16

Replica 0 · data A



Replica 1 · data B



DP synchronizes matching parameter shards across replicas

What does each method move?

Method	Split	Exchange	Main limit
DP	Samples	Parameter gradients	Sync + batch size
PP	Layers	Activations + their gradients	Bubbles + imbalance
TP	Within a layer	Partial results / activations	Frequent collectives

ZeRO (FSDP) additionally shards stored training state

Design a 16-GPU setup

Two servers, eight GPUs per server

Fast links inside each server

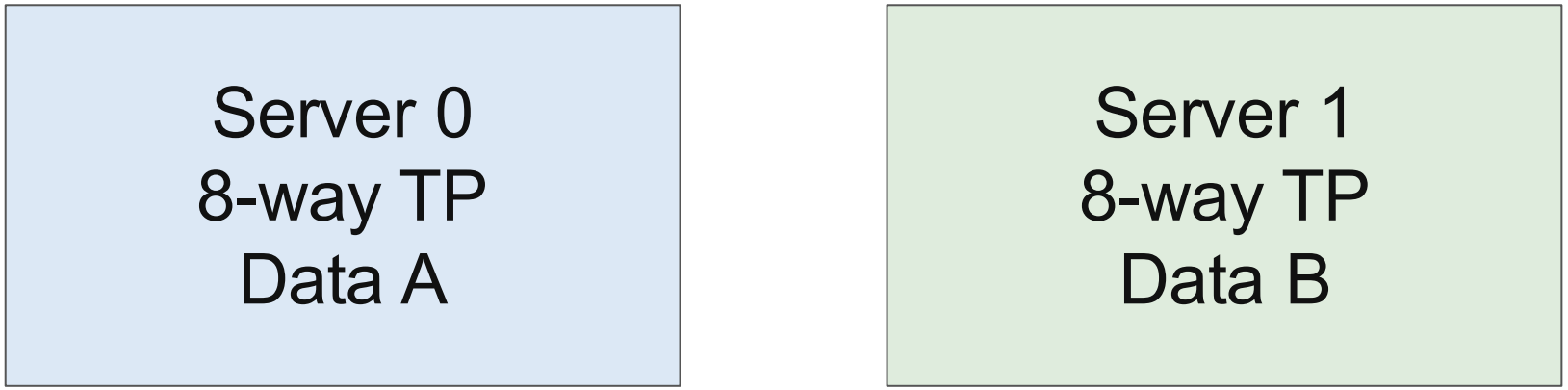
Slower links between servers

Training state fits across one server with TP

Choose (d, p, t). What crosses the slow link?

Assume enough batch parallelism for two replicas.

One reasonable first choice



DP gradient synchronization

d = 2, p = 1, t = 8

A starting configuration to profile, not a universal optimum.

Three questions to remember

1. What does one GPU own?
2. What must move between GPUs?
3. Where does time or memory go?

DP: examples PP: layers TP: within a layer